

Dorm AI

Product Requirements Document

Course: CSCI 387

Project: Dorm AI – Interactive Dorm Room Inspection System

Team 1: Andrea Gonzalez, Bryson Henderson, Natalie Greer, Julien Bourgeois (Team Lead)

Meeting Time: Thursdays at 1:15 PM

Communication: iMessage Group Chat

Document Version: Milestone 2 Draft

Date: March 2026

Document Purpose

This Product Requirements Document defines the first-release scope, core workflows, data model, user roles, non-functional requirements, and long-term direction for Dorm AI. Its purpose is to align the team around a realistic Minimum Viable Product while still documenting the broader product vision. The document is intentionally detailed enough to support design and implementation decisions, but constrained enough to remain agile and adaptable as the project evolves.

Dorm AI is a web-based, role-driven dorm room inspection platform designed to replace paper-based inspections and outdated legacy tools with a modern, secure, cloud-based system. It improves transparency between housing administrators, inspectors, and tenants by combining structured checklists, media evidence, and AI-assisted summaries into a centralized inspection workflow.

1. Goals

1.1 Purpose of the Project

The purpose of Dorm AI is to digitize the dorm room inspection process for universities and student housing operations. Many institutions still depend on paper forms, spreadsheets, or outdated CRUD systems that make inspections slow to complete, difficult to audit, and

vulnerable to disputes. Dorm AI aims to replace that fragmented process with a centralized platform that standardizes inspections, improves documentation quality, and gives all relevant parties better visibility into the condition of dorm rooms.

1.2 Problem Statement

Current dorm inspection workflows create several operational problems:

- inspection processes are inconsistent across inspectors
- documentation quality is often poor or incomplete
- media evidence is either missing or stored separately from reports
- tenants have limited transparency into what was recorded about their room
- administrators struggle to manage inspections, buildings, users, and disputes in one place
- manual workflows waste time and increase the likelihood of missed checklist items or contested damage claims

Dorm AI addresses these issues by giving administrators a system to manage housing data and inspection schedules, giving inspectors a structured digital workflow, and giving tenants visibility into completed reports and supporting evidence.

1.3 Product Vision

Dorm AI is intended to become a production-style inspection platform for university housing operations. The long-term vision is not only to digitize inspection workflows, but to create an intelligent platform that supports multimedia documentation, AI-assisted reporting, damage analysis, and scalable multi-campus deployment.

In the first version, however, the focus is narrow: deliver a reliable inspection system that works end to end and proves the concept.

1.4 MVP Statement

The minimum viable product is a secure, role-based web application that allows university housing administrators to manage buildings, rooms, users, and inspections; allows inspectors to complete digital room inspections with checklist items and photo evidence; and allows tenants to review completed inspection reports and attached documentation through a transparent portal.

1.5 MVP Objectives

The MVP must accomplish the following:

- replace paper or outdated inspection workflows with a digital process
- centralize university, building, room, and user management
- support role-based access for administrators, inspectors, and tenants
- allow administrators to schedule inspections
- allow inspectors to complete standardized checklists and upload photos
- generate an AI-written summary at the end of an inspection
- allow tenants to review completed inspection reports and evidence
- store data and media securely in the cloud

1.6 What Success Looks Like for Version 1

The MVP will be considered successful if:

- an administrator can create the housing structure and schedule inspections
- an inspector can complete a full inspection without using paper
- a tenant can log in and review a completed report with evidence
- inspection data is stored consistently and securely
- the system reduces ambiguity around what was inspected and what damage was recorded
- the product feels coherent, production-minded, and technically defensible as a full-stack system

1.7 MVP Scope Boundaries

To keep the project realistic for a class deliverable, the following items are explicitly **out of scope for the MVP** and are deferred to later iterations:

- online payment processing for damage charges
- video-based inspections as a required feature
- automatic computer vision damage detection
- real-time live AI assistant during an active inspection
- advanced analytics dashboards
- tenant dispute workflow automation
- multi-campus enterprise administration beyond the foundational data model

These may be referenced in future iterations, but they should not be treated as required Version 1 deliverables.

2. User Personas

Dorm AI has three primary user groups. These personas represent the major stakeholders whose workflows the system must support.

2.1 Persona 1 – Inspector

Name: James Smith

User Type: Inspector

Age: 20

Role: Resident Assistant for male dorms

Location: Jackson, Mississippi

Education Goal: Bachelor of Science in Biology

Background

James is responsible for carrying out routine dorm inspections while balancing academics and other campus obligations. He may inspect many rooms in one session and needs a workflow that is fast, structured, and reliable.

Pain Points

- paper inspections are slow and repetitive
- it is easy to forget checklist items
- weak documentation can create disputes later
- keeping track of which rooms are complete is inefficient
- writing summaries manually takes time

Goals

- complete inspections faster
- avoid missing required checks
- attach visual proof when damage is found
- clearly distinguish completed versus pending rooms
- reduce the time spent writing reports

How Dorm AI Supports Him

Dorm AI provides James with a guided checklist workflow, clear room-by-room status tracking, media upload support, and AI-generated summary assistance after submission. The product reduces friction while improving consistency and evidence quality.

2.2 Persona 2 – Tenant

Name: Stacy Moore
User Type: Tenant
Age: 18
Role: Student resident
Location: Nashville, Tennessee
Education Goal: Bachelor of Arts in Psychology

Background

Stacy is a first-year student living in campus housing. She wants clarity, fairness, and confidence that any damage associated with her room is documented accurately and transparently.

Pain Points

- fear of being charged for pre-existing damage
- limited access to inspection evidence
- uncertainty about how decisions are made
- stress around possible disputes and billing outcomes

Goals

- confirm that room condition is documented accurately
- access completed inspections easily
- see evidence supporting damage claims
- understand what has been recorded about her room
- feel that the process is fair and transparent

How Dorm AI Supports Her

Dorm AI gives Stacy a read-only portal where she can view inspection history, see attached images, review summaries, and understand what was documented about her dorm room. This improves trust and reduces uncertainty.

2.3 Persona 3 – Administrator

Name: Mathew Jackson
User Type: Administrator
Age: 41
Role: University Housing Administrator
Location: Oxford, Mississippi
Education: PhD in Data Science

Background

Mathew manages housing operations at the university level. He needs broad control over users, rooms, buildings, and inspection activity. He also needs a platform that can scale operationally as the housing system grows.

Pain Points

- managing buildings and room inventories across a housing system is cumbersome
- access control can become messy across different user roles
- inconsistent reports make oversight difficult
- disputes are harder to resolve when evidence is poor
- legacy systems do not provide strong operational visibility

Goals

- manage buildings, rooms, and users centrally
- schedule and monitor inspections efficiently
- enforce secure access permissions
- maintain reliable and auditable records
- standardize reporting quality across inspectors

How Dorm AI Supports Him

Dorm AI gives Mathew a centralized administrative interface for creating housing structures, managing users, scheduling inspections, and reviewing completed reports. It also enforces role-based access and lays the foundation for future reporting and analytics.

3. User Stories

The user stories below are written at the epic level and focus on must-have capabilities for the MVP.

3.1 Tenant User Stories

- As a tenant, I want to receive an invitation or join a valid housing profile so that I can access the platform securely.
- As a tenant, I want to log in and view my dorm assignment so that I know the system is tied to my correct room.

- As a tenant, I want to view upcoming inspections so that I know when my room will be inspected.
- As a tenant, I want to review completed inspection reports so that I understand what was recorded.
- As a tenant, I want to view attached photo evidence so that I can verify the condition findings.
- As a tenant, I want to see any listed damages connected to my room so that I understand the current condition history.
- As a tenant, I want to access support, policy, and housing contact information so that I know what to do if I have questions.

3.2 Inspector User Stories

- As an inspector, I want to log in and see my assigned inspections so that I can plan my work efficiently.
- As an inspector, I want to view room and building context before starting an inspection so that I know what location I am evaluating.
- As an inspector, I want to complete a structured checklist so that I do not miss required inspection items.
- As an inspector, I want to upload photo evidence during an inspection so that damages are documented clearly.
- As an inspector, I want to add notes to checklist items so that unusual conditions can be explained.
- As an inspector, I want to report new damage clearly so that the system captures it in a standardized way.
- As an inspector, I want the system to generate an AI summary after submission so that reporting is faster and more consistent.
- As an inspector, I want to see which inspections are complete, in progress, or pending so that I can manage my workflow.

3.3 Administrator User Stories

- As an administrator, I want to create and manage universities, buildings, and rooms so that the system matches the real housing structure.
- As an administrator, I want to create and manage users so that the correct people can access the correct records.
- As an administrator, I want to assign tenant and inspector roles so that permissions are enforced properly.
- As an administrator, I want to schedule inspections so that compliance and room checks occur on time.
- As an administrator, I want to review completed inspections so that I can oversee housing operations.

- As an administrator, I want to modify records when appropriate so that errors can be corrected by authorized staff.
- As an administrator, I want to view high-level operational data such as counts of buildings, rooms, users, and inspections so that I can understand current activity.
- As an administrator, I want to invite users into the platform so that onboarding can be controlled and secure.

3.4 Shared System User Stories

- As any authenticated user, I want the system to enforce permissions consistently so that I only see data relevant to my role and organization.
 - As any authenticated user, I want the platform to save and load data reliably so that I can trust the state of the system.
 - As any authenticated user, I want the interface to work on modern browsers and mobile-sized screens so that I can access it in realistic campus conditions.
-

4. ERD / Data Model

Dorm AI uses a relationally structured logical data model implemented on top of Firestore collections and related documents. The purpose of the model is to support multi-role access, scoped university data, inspection history, media evidence, and future extensibility.

4.1 Core Entities

4.1.1 Users

Represents an authenticated person in the system.

Attributes

- id
- name
- email
- phone
- gender
- bornAt
- createdAt
- updatedAt

A user is a person-level identity. Their role is not stored directly as a single global field because the same person could theoretically belong to different universities with different permissions. That role relationship is modeled through Memberships.

4.1.2 Universities

Represents a top-level housing organization using Dorm AI.

Attributes

- id
- name
- slug
- createdAt
- updatedAt

A university is the top organizational scope for security, membership, buildings, rooms, and inspections.

4.1.3 Memberships

Represents the relationship between a User and a University.

Attributes

- id
- userId
- universityId
- role
- status
- roomId
- classYear
- graduationYear
- employeeId
- assignedBuildingIds
- title
- createdAt
- updatedAt

Role values

- ADMIN

- INSPECTOR
- TENANT

Status values

- INVITED
- ACTIVE
- INACTIVE

This entity is critical because it supports:

- role-based access
 - tenant-to-room assignment
 - inspector-to-building assignment
 - future multi-university membership
-

4.1.4 Buildings

Represents a dorm building within a university.

Attributes

- id
- universityId
- name
- code
- address
- createdAt
- updatedAt

A university can have many buildings.

4.1.5 Rooms

Represents an inspectable dorm room.

Attributes

- id
- universityId
- buildingId
- number

- floor
- capacity
- createdAt
- updatedAt

A building can have many rooms. Tenant assignment is not stored directly on the room as the system uses Memberships for that relationship.

4.1.6 Inspections

Represents a scheduled or completed inspection event.

Attributes

- id
- universityId
- roomId
- inspectorId
- type
- status
- scheduledFor
- startedAt
- completedAt
- aiSummary
- createdBy
- createdAt
- updatedAt
- roomLabel
- tenantIds

Type values

- MOVE_IN
- ROUTINE
- MOVE_OUT

Status values

- SCHEDULED
- IN_PROGRESS
- COMPLETED
- CANCELED

This entity is the core operational record of the system.

4.1.7 InspectionItems

Represents each checklist item completed during an inspection.

Attributes

- id
- inspectionId
- section
- prompt
- response
- notes
- severity
- createdAt
- updatedAt

Response values

- GOOD
- FAIR
- DAMAGED
- NA

This structure allows inspections to be broken into structured, auditable sub-records.

4.1.8 Media

Represents uploaded inspection evidence.

Attributes

- id
- inspectionId
- inspectionItemId
- type
- storagePath
- uploadedBy
- createdAt

Type values

- PHOTO

- VIDEO
- AUDIO

For the MVP, photo evidence is required as the primary media type. Video and audio are allowed structurally in the model for future growth, but are not required MVP workflows.

4.1.9 Charges

Represents damage-related monetary records tied to an inspection.

Attributes

- id
- inspectionId
- inspectionItemId
- title
- amountCents
- status
- createdBy
- createdAt
- updatedAt

Status values

- PROPOSED
- ACCEPTED
- DISPUTED
- PAID
- WAIVED

This entity is useful even if full payment collection is deferred, because it allows the system to represent damage costs and their states over time.

4.2 Relationship Summary

- One User can have many Memberships.
- One University can have many Memberships.
- One University can have many Buildings.
- One Building can have many Rooms.
- One Room can have many Inspections.
- One Inspection belongs to one Room.

- One Inspection belongs to one Inspector.
- One Inspection can have many InspectionItems.
- One Inspection can have many Media records.
- One Inspection can have many Charges.
- One InspectionItem can optionally have related Media and Charges.

4.3 Textual ERD

Users

- Memberships
- Universities
- Buildings
- Rooms
- Inspections
- InspectionItems
- Media / Charges

More explicitly:

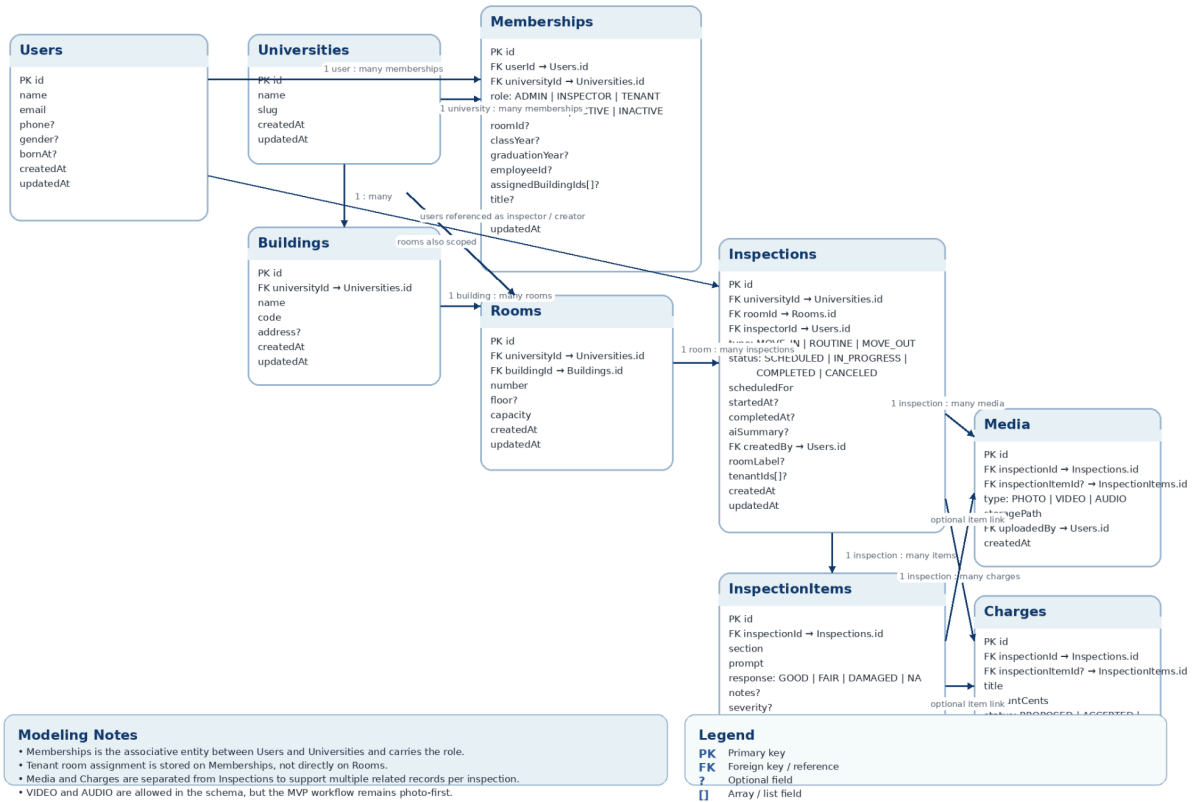
- Users 1..* Memberships
- Universities 1..* Memberships
- Universities 1..* Buildings
- Buildings 1..* Rooms
- Rooms 1..* Inspections
- Inspections 1..* InspectionItems
- Inspections 1..* Media
- Inspections 1..* Charges

4.4 Data Modeling Notes

- The model is intentionally university-scoped to support future multi-campus deployment.
- Memberships avoid hard-coding a single role onto the User entity.
- InspectionItems are normalized so the checklist can expand over time.
- Media is separated from Inspections to support multiple evidence assets.
- Charges are modeled separately from InspectionItems because not every finding becomes a financial record, and one inspection can generate multiple independent charges.

Dorm AI — Visual ERD

Based only on the entities, attributes, and relationships defined in the PRD.



5. Sitemap

The sitemap below represents the logical page hierarchy for the web application. Final route names may change during implementation, but the navigation structure should remain substantially similar.

5.1 Public Pages

- /
 - Landing / hero page
 - Product overview
 - Sign up CTA
 - Log in CTA
- /login
- /forgot-password

- `/privacy-policy`
- `/terms-of-use`

5.2 Onboarding and Setup

- `/setup`
 - role selection
 - profile setup
- `/join`
 - join via invite code or invitation link
- `/admin/create-organization`
 - create first university / housing organization

5.3 Administrator Application Area

- `/admin`
 - admin root / organization selector
- `/admin/[universityId]`
 - property home / overview dashboard
- `/admin/[universityId]/tenants`
- `/admin/[universityId]/tenants/[tenantId]`
- `/admin/[universityId]/inspectors`
- `/admin/[universityId]/inspectors/[inspectorId]`
- `/admin/[universityId]/inspections`
- `/admin/[universityId]/inspections/[inspectionId]`
- `/admin/[universityId]/inspections/schedule`
- `/admin/[universityId]/buildings`
- `/admin/[universityId]/buildings/[buildingId]`
- `/admin/[universityId]/rooms`
- `/admin/[universityId]/rooms/[roomId]`
- `/admin/[universityId]/settings`

5.4 Tenant Application Area

- `/tenant`
 - tenant home
- `/tenant/inbox`
- `/tenant/inspections`
- `/tenant/inspections/[inspectionId]`

- `/tenant/room`
- `/tenant/settings`

5.5 Inspector Application Area

- `/inspector`
 - inspector home
- `/inspector/inbox`
- `/inspector/inspections`
- `/inspector/inspections/[inspectionId]`
- `/inspector/inspections/[inspectionId]/start`
- `/inspector/inspections/[inspectionId]/review`
- `/inspector/settings`

5.6 Shared Utility Areas

- `/profile`
- `/notifications`
- `/support`
- `/unauthorized`
- `/404`

5.7 Navigation Note

Our notes use both “property” and “university” language. To keep the PRD internally consistent, this document treats **University** as the top-level organization in the data model and the admin dashboard. The UI may still use friendlier labels such as “Property” during onboarding, but implementation should map that concept back to the University / Building / Room hierarchy.

6. Page Descriptions

6.1 Public / Authentication Pages

6.1.1 Landing Page

Purpose: Introduce Dorm AI and serve as the primary sign-up entry point.

Primary Users: New administrators, invited users, evaluators.

Key Elements: hero section, platform description, sign-up form, log-in link, privacy policy, terms of use.

Primary Actions: create account, navigate to log in, review legal pages.

6.1.2 Log In Page

Purpose: Authenticate existing users.

Primary Users: Admins, inspectors, tenants.

Key Elements: email field, password field, submit button, forgot password link.

Primary Actions: log in, navigate to password reset.

6.1.3 Forgot Password Page

Purpose: Allow users to request a password reset email.

Primary Users: Existing users.

Key Elements: email field, reset request button, success/error state messaging.

Primary Actions: submit reset request.

6.2 Setup and Onboarding Pages

6.2.1 Account Setup Page

Purpose: Collect the minimum information required to route a user into the correct role-based experience.

Primary Users: Newly registered users.

Key Elements: role selection, name, contact info, optional demographic/profile fields, next-step CTA.

Primary Actions: select role, save profile.

6.2.2 Join Organization / Invite Page

Purpose: Allow tenants and inspectors to join the correct university context through an invitation flow.

Primary Users: Tenants and inspectors.

Key Elements: invite code input, invitation/deep-link handling, confirmation state, property/university lookup.

Primary Actions: join organization, validate invitation.

6.2.3 Admin Create First Organization Page

Purpose: Help a new administrator create the first university / housing organization after onboarding.

Primary Users: New admins.

Key Elements: organization name, slug, initial building prompt, confirmation CTA.

Primary Actions: create organization and continue to admin dashboard.

6.2.4 Blank Admin Dashboard State

Purpose: Provide a clear first-run experience when no buildings, rooms, or inspections exist yet.

Primary Users: New admins.

Key Elements: onboarding checklist, "Create your first building" CTA, "Invite users" CTA, empty-state guidance.

Primary Actions: begin setup tasks.

6.3 Administrator Pages

6.3.1 Admin Root / Organization Selector

Purpose: Let an administrator access one or more managed organizations.

Primary Users: Admins.

Key Elements: organization cards, quick stats, create organization button, profile controls.

Primary Actions: open organization dashboard, create organization.

6.3.2 Admin Home Dashboard

Purpose: Provide a high-level operating view of a selected organization.

Primary Users: Admins.

Key Elements: total buildings, total rooms, total tenants, total inspectors, upcoming inspections, recently completed inspections, summary charts, alerts.

Primary Actions: navigate to management sections, inspect current workload.

6.3.3 Tenants Page

Purpose: Manage tenant records within a university.

Primary Users: Admins.

Key Elements: searchable tenant table, room assignment, status, invite action, detail navigation, create/edit/delete actions.

Primary Actions: add tenant, edit tenant, assign room, invite tenant.

6.3.4 Tenant Detail Page

Purpose: Show an individual tenant's profile and housing context.

Primary Users: Admins.

Key Elements: tenant info, room assignment, membership status, inspection history, current charges, notes.

Primary Actions: edit tenant, reassign room, review inspection history.

6.3.5 Inspectors Page

Purpose: Manage inspector records and assignments.

Primary Users: Admins.

Key Elements: searchable inspector table, assigned buildings, status, contact info, detail navigation.

Primary Actions: add inspector, edit inspector, assign buildings, invite inspector.

6.3.6 Inspector Detail Page

Purpose: Show one inspector's profile and inspection workload.

Primary Users: Admins.

Key Elements: inspector info, assigned buildings, scheduled inspections, completed inspections, activity summary.

Primary Actions: update assignments, review performance context.

6.3.7 Inspections Page

Purpose: Central management surface for all inspections in the university.

Primary Users: Admins.

Key Elements: inspection table, filters by status/type/building/inspector, schedule button, detail navigation.

Primary Actions: create inspection, reschedule, cancel, open detail record.

6.3.8 Schedule Inspection Page / Modal

Purpose: Create a new inspection.

Primary Users: Admins.

Key Elements: room selector, inspection type, inspector selector, scheduled date/time, notes.

Primary Actions: save scheduled inspection.

6.3.9 Inspection Detail Page

Purpose: Review the full content of a scheduled or completed inspection.

Primary Users: Admins.

Key Elements: room info, inspector info, checklist results, notes, media, AI summary, damages/charges, timestamps.

Primary Actions: review, edit authorized fields, export or reference evidence.

6.3.10 Buildings Page

Purpose: Manage the dorm-building inventory for the university.

Primary Users: Admins.

Key Elements: building list, codes, addresses, room counts, create/edit/delete actions.

Primary Actions: create building, update building, view rooms.

6.3.11 Rooms Page

Purpose: Manage inspectable rooms.

Primary Users: Admins.

Key Elements: room list, building, floor, capacity, assigned tenant, current status.

Primary Actions: create room, edit room, assign tenant, open room record.

6.3.12 Settings Page

Purpose: Control organization-level settings.

Primary Users: Admins.

Key Elements: organization profile, user permissions settings, notification preferences, checklist template settings, legal/support info.

Primary Actions: update configuration.

6.4 Tenant Pages

6.4.1 Tenant Home

Purpose: Give tenants a concise overview of room-related activity.

Primary Users: Tenants.

Key Elements: upcoming inspections, recent completed inspections, open alerts, latest damages or room-condition notes.

Primary Actions: open inspection details, navigate to room information.

6.4.2 Tenant Inbox

Purpose: Centralize user-facing notifications.

Primary Users: Tenants.

Key Elements: reminders, inspection completion notices, system alerts.

Primary Actions: open linked records.

6.4.3 Tenant Inspections Page

Purpose: Show all upcoming and past inspections relevant to the tenant.

Primary Users: Tenants.

Key Elements: date, type, status, summary preview, detail links.

Primary Actions: review inspection records.

6.4.4 Tenant Inspection Detail Page

Purpose: Provide full transparency into a completed inspection.

Primary Users: Tenants.

Key Elements: room data, checklist outcomes, inspector notes, attached media, AI summary, related damages.

Primary Actions: review evidence and findings.

6.4.5 Tenant Room / Property Info Page

Purpose: Show the tenant's assigned room and housing context.

Primary Users: Tenants.

Key Elements: building name, room number, contact information, current condition notes, historical damage context.

Primary Actions: view information, access support contacts.

6.4.6 Tenant Settings Page

Purpose: Let tenants manage account-level settings and support access.

Primary Users: Tenants.

Key Elements: profile, password management, privacy links, terms, support request link, sign out.

Primary Actions: update profile and preferences.

6.5 Inspector Pages

6.5.1 Inspector Home

Purpose: Give inspectors a practical daily overview.

Primary Users: Inspectors.

Key Elements: assigned buildings, today's inspections, in-progress inspections, overdue items, quick-start actions.

Primary Actions: start or resume inspections.

6.5.2 Inspector Inbox

Purpose: Show reminders and assignment notifications.

Primary Users: Inspectors.

Key Elements: scheduled inspection notices, reassignment alerts, completion confirmations.

Primary Actions: open inspection links.

6.5.3 Inspector Inspections Page

Purpose: Provide a working queue for inspection execution.

Primary Users: Inspectors.

Key Elements: filters for scheduled/in-progress/completed, building/room labels, dates, status indicators.

Primary Actions: start, resume, or review inspections.

6.5.4 Inspection Execution Page

Purpose: Act as the main inspection workspace.

Primary Users: Inspectors.

Key Elements: room header, checklist sections, response inputs, notes, image upload, existing damage reference, progress indicator, submit button.

Primary Actions: complete checklist items, report damage, attach images, save progress, finish inspection.

6.5.5 New Damage Reporting Interface

Purpose: Capture detailed evidence when new damage is found.

Primary Users: Inspectors.

Key Elements: damage type, location, description, severity, photo upload, optional note.

Primary Actions: create structured damage record.

6.5.6 Inspection Review / Submit Page

Purpose: Present a final summary before completion.

Primary Users: Inspectors.

Key Elements: checklist review, uploaded evidence review, AI-generated summary preview, confirm submission action.

Primary Actions: finalize inspection.

6.5.7 Inspector Settings Page

Purpose: Manage inspector account settings.

Primary Users: Inspectors.

Key Elements: profile info, password reset, notifications, sign out.

Primary Actions: update preferences.

7. Wireframes (Optional but Recommended)

Because actual image wireframes are optional, the following low-fidelity layout descriptions define what should appear on the main screens and where.

7.1 Authentication Funnel Wireframe

Landing Page Layout

- top area: logo, navigation, log-in link
- center hero: Dorm AI headline, short description, sign-up form
- lower section: brief feature explanations
- footer: privacy policy, terms of use

Log In Page Layout

- centered auth card
- email input
- password input
- forgot password link
- submit button

Forgot Password Layout

- single-field recovery form
 - email input
 - reset button
 - confirmation message
-

7.2 Admin Dashboard Wireframe

Overall Layout

- left sidebar: navigation tabs
- top navigation: organization switcher, search, notifications, profile
- main content area: cards, tables, charts, detail panels
- optional right rail: contextual actions or help

Main Home View

- top summary cards: total buildings, rooms, tenants, inspectors, inspections
- middle row: upcoming inspections and recent activity
- lower section: operational charts and quick actions

This should resemble a production-style operations dashboard rather than a consumer app.

7.3 Tenant Dashboard Wireframe

Layout

- mobile-first responsive design
- top header with room and building info
- tab navigation or sidebar depending on viewport
- home view with alerts and recent inspections
- detail pages that emphasize transparency and read-only evidence access

The tenant experience should feel simple, clear, and non-technical.

7.4 Inspector Dashboard Wireframe

Layout

- action-first interface
- prominent inspection queue
- strong status indicators for scheduled, in-progress, and completed work
- minimal clutter so inspections can be completed quickly from a phone or laptop

Inspection Runner Layout

- sticky header with room and progress
 - checklist items grouped by section
 - inline note and image upload controls
 - bottom action bar for save, next section, finish inspection
-

7.5 Empty-State Wireframes

Admin Empty State

- “You do not have any buildings or rooms yet”
- CTA to create first building
- CTA to invite first users
- setup checklist

Tenant Empty State

- “No inspections available yet”
- supportive message and room info

Inspector Empty State

- “No assigned inspections right now”
- upcoming schedule or refresh state

For fun, we have also included images of potential wireframes & what they will look like. But of course, projects evolve as teams start to work on them.

Dorm AI — Wireframe Pack v2

Admin is web-only. Tenant and Inspector are mobile app only. Revised to match your direction.

Included screens

• Admin web (12): root property selector, empty state, overview, buildings list, building detail, rooms list, tenants list, tenant detail, inspectors list, inspector detail, inspections list + schedule, inspection detail.

• Mobile app (8): tenant home, tenant inspections, tenant inspection detail, tenant dorm / property info, inspector home, inspector inspections, inspector runner, inspector damage + submit.

admin_01_root_property_selector.png

admin_02_empty_state_create_property.png

admin_03_overview_dashboard.png

admin_04_buildings_list.png

admin_05_building_detail.png

admin_06_rooms_list.png

admin_07_tenants_list.png

admin_08_tenant_detail.png

admin_09_inspectors_list.png

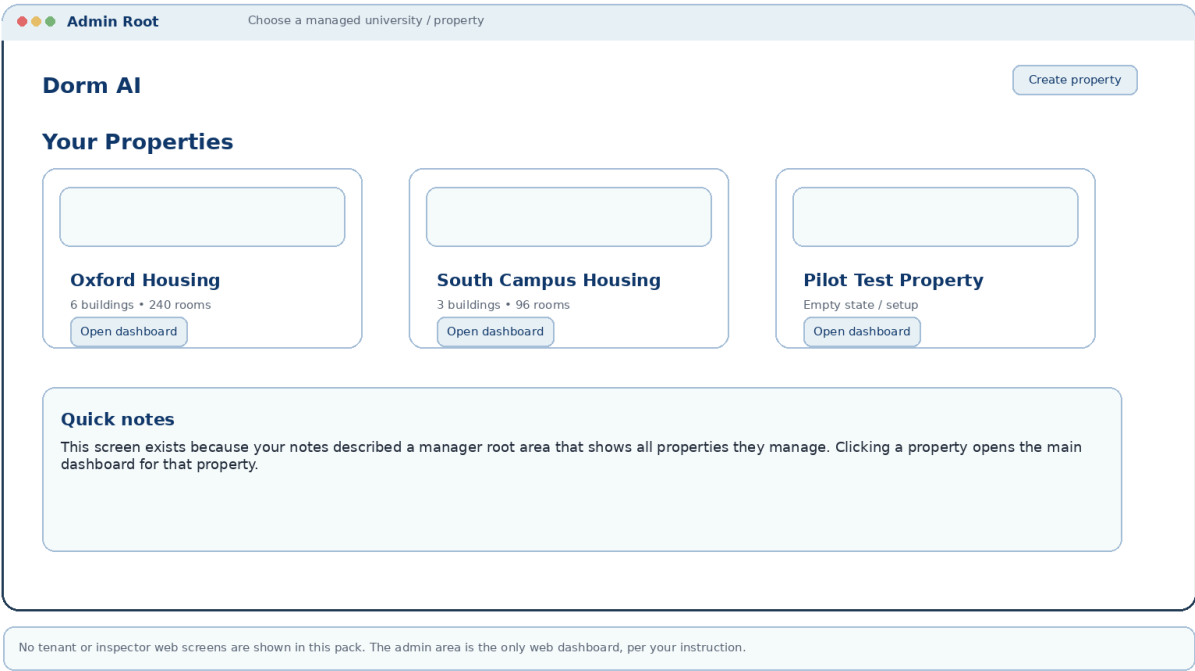
admin_10_inspector_detail.png

This revision fixes the platform split: only admin uses desktop web, while tenant and inspector use mobile app flows.

admin_12_inspection_detail.png

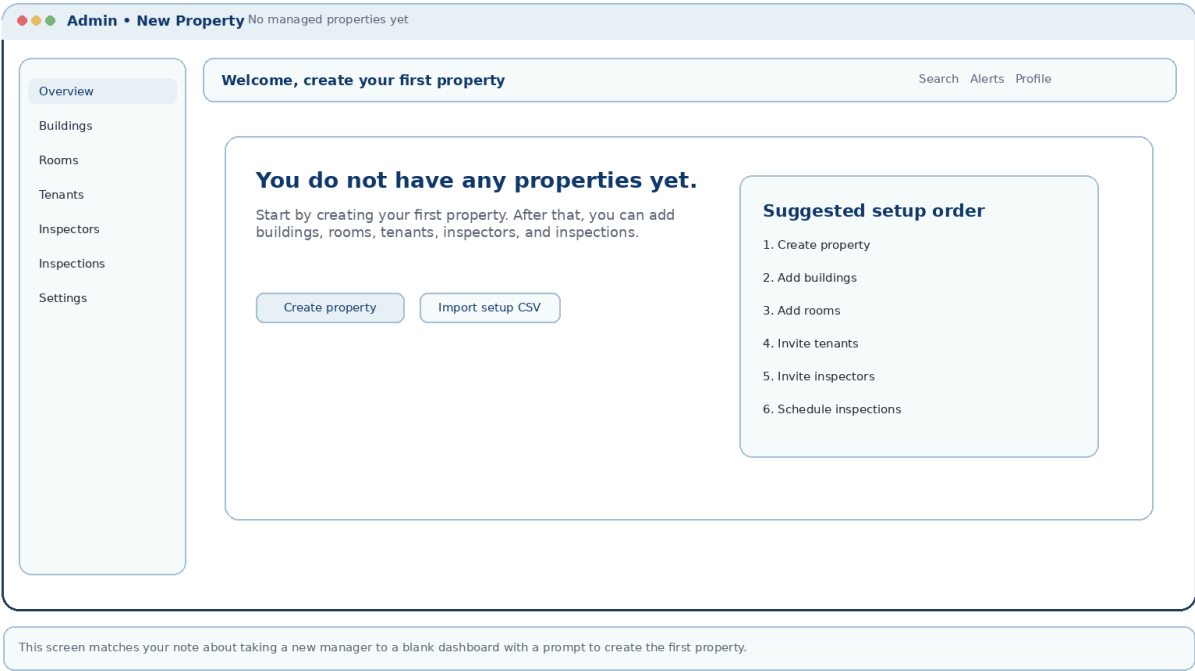
Dorm AI — Admin Web 01

Admin root page / organization selector. Web-only, based on the PRD and your notes.



Dorm AI — Admin Web 02

Empty-state dashboard after manager onboarding.



Dorm AI — Admin Web 03

Overview dashboard for a selected property.

Admin • Oxford Housing

Overview

Buildings

Rooms

Tenants

Inspectors

Inspections

Settings

Oxford Housing

Search Alerts Profile

Buildings

6

Rooms

240

Tenants

228

Inspectors

12

Upcoming

19

Completed

88

Inspection activity



Upcoming inspections

Room	Type	Inspector	Date	Status
A-201	Routine	James	Mar 20	Scheduled
B-114	Move-out	Ava	Mar 20	Scheduled
C-302	Routine	Noah	Mar 21	Scheduled
A-014	Move-in	James	Mar 21	Scheduled

Quick actions

Create building

Add room

Invite tenants

Invite inspectors

Schedule inspection

Notes

High-level counts and charts are allowed here. Advanced analytics beyond this are future

This stays inside the PRD: overview stats, upcoming items, quick actions, and light charts, not a full analytics suite.

Dorm AI — Admin Web 04

Buildings management page.

Admin • Oxford Housing

Buildings

Overview

Buildings

Rooms

Tenants

Inspectors

Inspections

Settings

Buildings

Search Import CSV Add building

Search

Code

Status

Buildings

Name	Code	Address	Rooms
Alpha Hall	ALP	101 University Ave	80
Beta Hall	BET	202 College Dr	64
Gamma Hall	GAM	303 Campus Way	96
Delta Hall	DEL	404 Dorm Loop	0

Selected building

Name: Alpha Hall

Code: ALP

Address: 101 University Ave

Rooms: 80

Inspectors assigned: 3

Edit

View rooms

Buildings are a core admin CRUD surface in the PRD and data model. This page is web because managers control the housing structure.

Dorm AI — Admin Web 05

Building detail page with room and inspection context.

Admin • Alpha Hall

Building detail

Overview

Buildings

Rooms

Tenants

Inspectors

Inspections

Settings

Alpha Hall

Search Alerts Profile

Building profile

Building: Alpha Hall

Total rooms: 80

Code: ALP

Inspectors assigned: James, Ava, Noah

Address: 101 University Ave

Rooms in this building

Room	Floor	Capacity	Tenant
A-201	2	2	Stacy Moore
A-202	2	2	Jaden Lee
A-203	2	2	Mila Patel
A-204	2	2	Open

Upcoming inspections

Inspection	Room	Date	Status
Routine	A-201	Mar 20	Scheduled
Move-out	A-204	Mar 22	Scheduled
Routine	A-215	Mar 25	Scheduled

Edit building

Add room

This detail page links the real housing structure to downstream room and inspection management, which is directly implied by the sitemap and CRUD notes.

Dorm AI — Admin Web 06

Rooms management page.

Admin • Oxford Housing

Rooms

Overview

Buildings

Rooms

Tenants

Inspectors

Inspections

Settings

Rooms

Search Bulk import Add room

Search room

Building

Floor

Occupancy

Rooms

Room	Building	Floor	Capacity	Tenant
A-201	Alpha Hall	2	2	Stacy Moore
A-202	Alpha Hall	2	2	Jaden Lee
B-114	Beta Hall	1	2	Open
C-302	Gamma Hall	3	2	Mila Patel

Selected room

Room: A-201

Building: Alpha Hall

Floor: 2

Capacity: 2

Assigned tenant: Stacy Moore

Edit room

View history

Room assignment exists in the data model and is operationally central to inspections, so the admin web area needs this dedicated page.

Dorm AI — Admin Web 07

Tenants management page.

Admin • Oxford Housing

Tenants

Overview

Buildings

Rooms

Tenants

Inspectors

Inspections

Settings

Tenants

Bulk invite CSV

Invite tenant

Search tenant

Building

Room

Status

Tenants

Name	Email	Room	Building	Status
Stacy Moore	stacy@school.edu	A-201	Alpha Hall	Active
Jaden Lee	jaden@school.edu	A-202	Alpha Hall	Invited
Mila Patel	mila@school.edu	C-302	Gamma Hall	Active
Owen Ross	owen@school.edu	B-114	Beta Hall	Inactive

Selected tenant

Name: Stacy Moore

Room: A-201

Building: Alpha Hall

Membership: Active

Upcoming inspection: Mar 20

Edit tenant

Open detail

Bulk invite appears here because your notes explicitly mentioned using CSV for tenant invites.

Dorm AI — Admin Web 08

Tenant detail page.

Admin • Tenant Detail

Stacy Moore

Overview

Buildings

Rooms

Tenants

Inspectors

Inspections

Settings

Tenant detail

Search Alerts Profile

Profile

Name: Stacy Moore

Email: stacy@school.edu

Role: Tenant

Room: A-201

Building: Alpha Hall

Housing context

Current room assignment: A-201

Existing room damages on file: 1

Most recent completed inspection: Mar 10

Next scheduled inspection: Mar 20

Inspection history

Inspection	Date	Status
Move-in	Aug 18	Completed
Routine	Mar 10	Completed
Routine	Mar 20	Scheduled

Damage / charge records

Charge / Damage	Amount	Status
Wall paint damage	\$35.00	Proposed

Reassign room

Resend invite

Modify record

This admin detail page is realistic to the PRD: profile, room assignment, inspection history, and damage/charge visibility. No payment collection UI was added.

Dorm AI — Admin Web 09

Inspectors management page.

Admin • Oxford Housing

Inspectors

Overview

Buildings

Rooms

Tenants

Inspectors

Inspections

Settings

Inspectors

Bulk invite CSV

Invite inspector

Search inspector

Assigned building

Status

Inspectors

Name	Email	Buildings	Scheduled	Status
James Smith	james@school.edu	Alpha, Beta	8	Active
Ava Reed	ava@school.edu	Gamma	6	Active
Noah Brooks	noah@school.edu	Alpha	3	Invited

Selected inspector

Name: James Smith

Assigned buildings: Alpha, Beta

Upcoming inspections: 8

Completed this week: 12

Edit access

Open detail

Inspectors are managed separately from tenants because the PRD assigns them different permissions and building scope.

Dorm AI — Admin Web 10

Inspector detail page.

Admin • Inspector Detail

James Smith

Overview

Buildings

Rooms

Tenants

Inspectors

Inspections

Settings

Inspector detail

Search

Alerts

Profile

Profile

Name: James Smith

Email: james@school.edu

Role: Inspector

Assigned buildings: Alpha, Beta

Status: Active

Update buildings

Resend invite

Assigned inspections

Room	Type	Date	Status
A-201	Routine	Mar 20	Scheduled
A-202	Routine	Mar 20	Scheduled
B-114	Move-out	Mar 20	In progress
B-116	Routine	Mar 21	Scheduled

Open inspection list

This page stays within the admin web scope: view inspector info, assignments, and the list of inspections tied to that inspector.

Dorm AI — Admin Web 11

Inspections page with schedule action.

Admin • Oxford Housing

Inspections

Overview

Buildings

Rooms

Tenants

Inspectors

Inspections

Settings

Inspections

Search Alerts Profile

Schedule new

Room

Type

Inspector

Status

Inspections

Room	Type	Inspector	Date	Status
A-201	Routine	James	Mar 20	Scheduled
B-114	Move-out	Ava	Mar 20	In progress
C-302	Routine	Noah	Mar 21	Scheduled
A-014	Move-in	James	Mar 22	Scheduled

Schedule inspection

Room

Inspection type

Inspector

Date / time

Create inspection

This is the realistic admin scheduling surface described in the PRD: room, type, inspector, date/time, and a list of all inspections.

Dorm AI — Admin Web 12

Inspection detail page.

Admin • Inspection Detail

Line inspection for room A-201

Overview

Buildings

Rooms

Tenants

Inspectors

Inspections

Settings

Inspection detail

Search Alerts Profile

Inspection overview

Room: A-201
Building: Alpha Hall
Type: Routine
Inspector: James Smith

Status: Completed
Scheduled: Mar 10 • 2:00 PM
Completed: Mar 10 • 2:23 PM

AI summary

The room was generally in acceptable condition. Damage was observed on the west wall near the desk area and documented with photo evidence. Remaining checklist items were completed without major issues.

Checklist

Section	Result
Furniture	Good
Walls / paint	Damaged
Floor	Fair
Safety / compliance	Good

Attached media

Photo

Photo

Photo

Photo

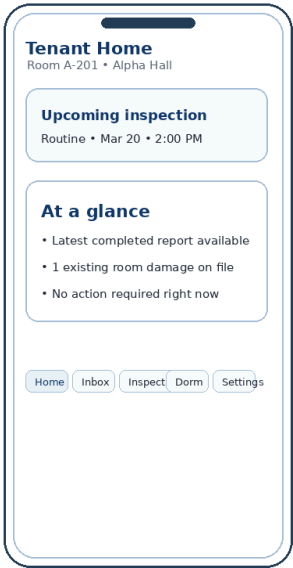
Modify record

Open room history

The admin detail view shows exactly what the PRD calls for: checklist, evidence, AI summary, timestamps, and room/inspector context.

Dorm AI — Mobile App 13

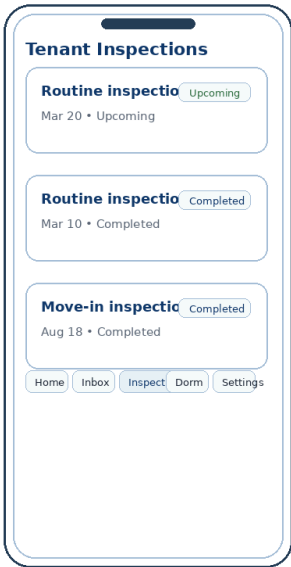
Tenant home screen. Tenant is mobile app only in this revised pack.



Tenant stays mobile-only here. No desktop tenant dashboard is included.

Dorm AI — Mobile App 14

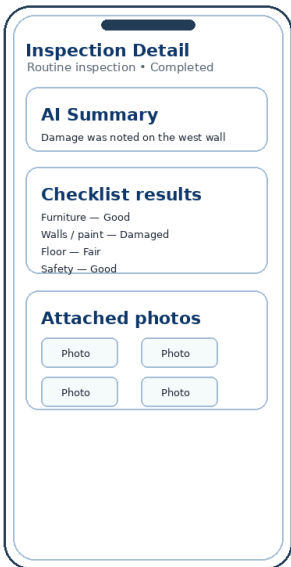
Tenant inspections list.



The tenant PRD explicitly includes a page for upcoming and past inspections with detail navigation. That is what this mobile screen shows.

Dorm AI — Mobile App 15

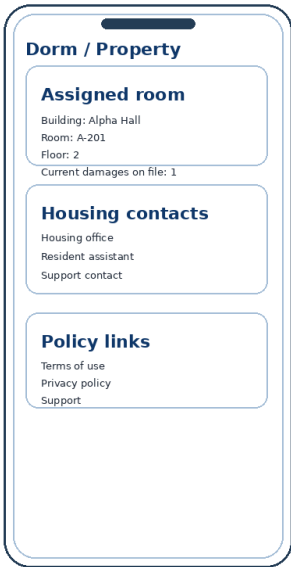
Tenant inspection detail.



This is intentionally read-only, matching the MVP direction: tenants review reports and evidence but do not operate inspections.

Dorm AI — Mobile App 16

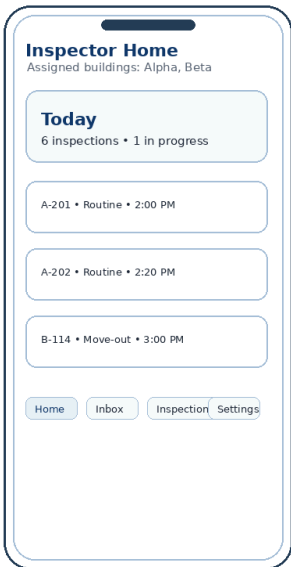
Tenant room / property information.



This mobile tenant screen is grounded in the PRD section about dorm/property information and settings/support access.

Dorm AI — Mobile App 17

Inspector home screen / queue.



Inspector is mobile-only here. The PRD emphasized quick access to assigned work and a simple action-first flow.

Dorm AI — Mobile App 18

Inspector inspections list.

My Inspections

A-201 • Routine

Scheduled

Mar 20 • 2:00 PM

B-114 • Move-out

In progress

Mar 20 • 3:00 PM

C-302 • Routine

Scheduled

Mar 21 • 4:00 PM

Resume in-progress inspection

This list is the realistic mobile entry into the inspection runner: scheduled, in-progress, and future inspections assigned by the admin.

Dorm AI — Mobile App 19

Inspector inspection runner.

Inspection Runner

Room A-201 • Progress 7 / 12

Furniture

GOOD

FAIR

DAMAGED

NA

Walls / paint

GOOD

FAIR

DAMAGED

NA

Floor

GOOD

FAIR

DAMAGED

NA

Add photo

View existing

Finish inspection

The runner remains simple and mobile-first: checklist, evidence, existing damages button, and finish inspection. No live AI assistant was added because that is future scope.

Dorm AI — Mobile App 20

Inspector damage reporting and final review.

The image displays two mobile app screens side-by-side. The left screen, titled 'Report New Damage', features a form with fields for 'Type', 'Location', and 'Description'. Below these are 'Photo' and 'Severity' fields, followed by 'Check if similar exists' and 'Create damage record' buttons. The right screen, titled 'Review / submit state', shows a checklist completion status of '12 / 12', 'Photos attached: 4', and 'New damage records: 1'. It also includes a note 'AI summary will generate after completion' and 'Back to checklist' and 'Submit' buttons.

This screen reflects your notes directly: report new damage with type, location, description, image/video field concept, similarity check, then a final review / submit step.

8. Non-Functional Requirements

8.1 Technical Stack Requirements

- The frontend must be built with Next.js.
- The backend must use Firebase services, including Firebase Authentication, Firestore, Cloud Storage, and Cloud Functions.
- AI summary generation must integrate with Google Vertex AI.
- The application must be deployed on Vercel.
- Source control and deployment must support a Git-based workflow.
- Email delivery must use AWS SES or an equivalent transactional email pipeline approved by the team.

8.2 Security Requirements

- The system must enforce role-based access control for Admin, Inspector, and Tenant roles.

- Firestore security rules must prevent users from accessing records outside their authorized university scope.
- Cloud Storage rules must restrict media visibility based on organization and role.
- All network traffic must occur over HTTPS.
- Sensitive records must only be readable by authorized users.
- Authentication state must be validated server-side or through trusted session handling where appropriate.

8.3 Performance Requirements

- The interface must remain usable on standard broadband and modern campus Wi-Fi.
- Page loads for standard dashboards should feel responsive under typical class-project data volume.
- Image uploads should complete reliably without breaking the inspection flow.
- The system must support concurrent usage by multiple users without data corruption or conflicting state.

8.4 Reliability Requirements

- Inspection progress should not be lost due to minor client-side interruptions.
- Errors should be surfaced clearly to the user.
- System actions that fail should produce recoverable feedback rather than silent failure.
- Logging should exist for critical backend operations, including scheduling and AI summary generation.

8.5 Scalability Requirements

- The data model must support multiple universities without requiring a redesign.
- University data should be logically partitioned to prevent cross-organization leakage.
- Media storage patterns should remain compatible with larger future volumes.
- The architecture should support future expansion into multi-campus deployment.

8.6 Usability Requirements

- The platform must be responsive across desktop and mobile browser sizes.
- The admin interface should be desktop-optimized.
- Tenant and inspector experiences should be mobile-friendly.
- Navigation should remain consistent across roles.
- Forms should prioritize clarity and low friction.

8.7 Browser Compatibility Requirements

- The application must function correctly in current versions of Chrome, Safari, Edge, and Firefox.

8.8 Maintainability Requirements

- The codebase should be modular and separated by feature domain where practical.
 - Reusable components should be used for tables, forms, auth flows, and status views.
 - Validation rules should be centralized where possible.
 - The system should be designed so that new checklist categories, buildings, roles, and future AI features can be added with limited refactoring.
-

9. Risks

9.1 AI Accuracy Risk

AI-generated summaries may be vague, inconsistent, or factually wrong if the source inputs are weak.

Mitigation:

Require human review before finalization. Treat AI output as assistive, not authoritative.

9.2 Data Privacy and Compliance Risk

The system handles student housing information and room-condition evidence, which can be sensitive.

Mitigation:

Use strict access rules, scoped data access, secure storage rules, and audit-oriented record design. Avoid exposing data outside role and university boundaries.

9.3 Role / Permission Risk

Incorrect authorization logic could expose tenant records, inspection evidence, or admin actions to the wrong users.

Mitigation:

Design around university-scoped memberships, role-based guards, and explicit permission checks in both frontend and backend.

9.4 Scope Creep Risk

Because the concept has strong real-world potential, it is easy to over-expand into payments, analytics, dispute resolution, or advanced AI too early.

Mitigation:

Anchor the project around the MVP statement and treat future features as clearly out of scope for the class milestone.

9.5 Media Storage and Cost Risk

Photo and future video evidence can quickly increase storage usage and complexity.

Mitigation:

Start with photo-first MVP, use structured storage paths, and keep video workflows out of MVP.

9.6 Adoption Risk

University staff may resist switching from familiar processes even if those processes are inefficient.

Mitigation:

Keep the workflow simple, emphasize time savings, and reduce the learning curve through straightforward screens and role-specific dashboards.

9.7 Technical Integration Risk

The project spans multiple services: Next.js, Firebase, Vertex AI, Vercel, and email delivery infrastructure.

Mitigation:

Keep integrations narrow in the MVP, define clear boundaries between modules, and prioritize end-to-end reliability over excessive feature count.

9.8 Data Consistency Risk

Inspection data, room assignments, and memberships could drift out of sync if updates are not modeled carefully.

Mitigation:

Use normalized references, scoped updates, and predictable creation/edit flows.

10. Future Iterations

10.1 Video-Based Inspections

Inspectors will be able to upload video walkthroughs in addition to photos. This would improve evidence quality and allow richer inspection context.

10.2 Automated Damage Detection

Computer vision models could analyze uploaded media to identify likely damage categories such as wall damage, stains, broken fixtures, or furniture wear.

10.3 Damage Cost Assistance

The platform could generate suggested repair cost ranges based on the type and severity of damage, helping administrators standardize estimates.

10.4 Live AI Assistant

Inspectors could receive real-time AI guidance during inspections, including checklist suggestions, note drafting, and policy clarification.

10.5 Tenant Dispute Workflow

Tenants could formally dispute a damage claim, submit counter-evidence, and track review status within the platform.

10.6 Payment Collection

Charges could eventually connect to a secure payment system so that tenants can resolve approved balances directly through the portal.

10.7 Analytics Dashboard

Administrators could gain deeper reporting such as completion rates, recurring damage patterns, high-risk buildings, and inspection throughput.

10.8 Multi-Campus / Enterprise Scaling

Dorm AI could expand into a white-labeled multi-campus platform with centralized enterprise administration and institution-specific branding.

11. Functional Requirements

This section captures required system behavior not fully expressed by the user stories alone.

11.1 Authentication and Account Management

- FR-1** The system shall allow new users to create an account with email and password.
- FR-2** The system shall allow existing users to log in with email and password.
- FR-3** The system shall allow users to request a password reset email.
- FR-4** The system shall support onboarding flows that route users based on selected role.
- FR-5** Tenants and inspectors shall be able to join an organization using an invitation flow, code, or deep link.
- FR-6** Administrators shall be able to create an initial organization during setup.

11.2 User and Organization Management

- FR-7** Administrators shall be able to create, edit, and deactivate tenant accounts.
- FR-8** Administrators shall be able to create, edit, and deactivate inspector accounts.
- FR-9** Administrators shall be able to assign users to the correct university scope.
- FR-10** Administrators shall be able to assign tenants to rooms.
- FR-11** Administrators shall be able to assign inspectors to buildings or inspection responsibilities.

11.3 Building and Room Management

- FR-12** Administrators shall be able to create, edit, and delete buildings.
- FR-13** Administrators shall be able to create, edit, and delete rooms.
- FR-14** Rooms shall belong to a specific building and university.
- FR-15** The system shall preserve room-level inspection history over time.

11.4 Inspection Scheduling and Oversight

FR-16 Administrators shall be able to schedule inspections for a room.

FR-17 An inspection shall store type, status, schedule, assigned inspector, room, and audit timestamps.

FR-18 Administrators shall be able to review scheduled, in-progress, completed, and canceled inspections.

FR-19 Inspectors shall only see inspections assigned to them or otherwise authorized for their scope.

11.5 Inspection Execution

FR-20 Inspectors shall be able to start an assigned inspection.

FR-21 Inspectors shall be able to complete checklist items using predefined response options.

FR-22 Inspectors shall be able to add notes to inspection items.

FR-23 Inspectors shall be able to upload photo evidence during an inspection.

FR-24 Inspectors shall be able to save progress and submit an inspection as completed.

FR-25 The system shall record start and completion timestamps.

11.6 AI Summary Generation

FR-26 The system shall generate an AI-written summary after inspection data is submitted.

FR-27 The AI summary shall be stored with the inspection record.

FR-28 The inspector or administrator shall be able to review the generated summary before relying on it operationally.

11.7 Tenant Transparency

FR-29 Tenants shall be able to view upcoming inspections related to their room.

FR-30 Tenants shall be able to view completed inspections tied to their room.

FR-31 Tenants shall be able to view photo evidence associated with completed inspections.

FR-32 Tenants shall be able to view damages or charge-related records connected to their room history, subject to permissions.

11.8 Notifications

FR-33 The system shall send email notifications for key account and inspection events.

FR-34 The system shall support reminder emails for scheduled inspections.

FR-35 The system shall support completion notifications when an inspection becomes available for review.

11.9 Audit and Integrity

FR-36 The system shall timestamp major create/update events.

FR-37 The system shall store the user responsible for creating inspections and uploading media where applicable.

FR-38 The system shall enforce university-level data boundaries in all major operations.

12. System Emails and Notifications

Including an email section helps the PRD better mirror a real product specification and follows the structure used in the attached example.

12.1 Account Invitation Email

Purpose: Invite a tenant or inspector to join Dorm AI.

Audience: Tenant or Inspector

Trigger: Admin sends invitation

Content: organization name, role context, join link or code, support contact

12.2 Password Reset Email

Purpose: Allow a user to recover account access.

Audience: Any user

Trigger: Forgot password request

Content: secure reset link, expiration note

12.3 Inspection Scheduled Email

Purpose: Notify relevant users about an upcoming inspection.

Audience: Tenant and Inspector

Trigger: Inspection scheduled or updated

Content: room, building, date/time, inspection type, link to platform

12.4 Inspection Reminder Email

Purpose: Reduce missed inspections and increase readiness.

Audience: Tenant and Inspector

Trigger: Scheduled reminder before inspection

Content: reminder timing, room/building context, expected action if any

12.5 Inspection Completed Email

Purpose: Inform the tenant and admin that a report is available.

Audience: Tenant and Admin

Trigger: Inspection marked completed

Content: summary notice, link to report, next-step guidance

12.6 Account Status Change Email

Purpose: Inform users when access changes materially.

Audience: Any user

Trigger: Account deactivation, invitation acceptance, or similar status transition

Content: status update, reason if appropriate, support pathway

13. Assumptions and Scope Notes

13.1 Delivery Assumptions

- Dorm AI is a web-first application for the MVP.
- Native mobile apps are not required for Milestone 2.
- Photo uploads are core MVP evidence; video is future-facing.
- AI is assistive for summaries, not a source of final truth.

13.2 Product Vocabulary Note

- “University” is the top organizational scope in the data model.
- “Building” represents a dorm building.
- “Room” represents the inspectable unit.
- The UI may use “property” language for friendliness, but implementation should remain consistent with the underlying model.

13.3 MVP Discipline Note

Several features discussed in brainstorming are valuable but should not be treated as Version 1 requirements: payments, live multimodal assistant, automated cost estimation, full dispute resolution, computer vision damage detection, and advanced analytics. Including them in the roadmap is appropriate; committing them to the MVP would weaken the document by making scope unrealistic.

14. Conclusion

Dorm AI is a production-minded, role-based dorm inspection platform that addresses a practical institutional problem: housing inspections are often slow, inconsistent, and poorly documented. By centralizing building and room management, inspection scheduling, structured checklist execution, media evidence, and tenant transparency in one web application, Dorm AI provides a realistic and technically meaningful solution.

The MVP is intentionally focused. It proves the core workflow from administration through inspection through tenant review, while leaving more ambitious automation and AI features for later versions. That makes the project stronger both academically and practically: it demonstrates clear scope control, full-stack architecture, thoughtful data modeling, and an extensible product strategy.